AI FOUNDATIONS

Neural networks and language models from first principles

Handbook

How the course works, how to work, the maths you need, machines for machine learning, and setup

1 The course

2 How to work

3 The maths you need

4 Machines for machine learning

5 Setup

6 Reference library

7 Progress tracker

First edition, October 2026

Contents

1 The course 2
What you will have built 2
Four units 3
Four threads 3
Who the course is for 4
The volumes and the code 4
Conventions 4
2 How to work 5
The anatomy of a chapter 5
Labs and done-when criteria 5
The notebook and the evidence 5
Running experiments 5
Measuring properly 6
Reading papers 6
Getting unstuck 6
3 The maths you need 7
Vectors and matrices 7
Derivatives and the chain rule 7
Probability 8
Logarithms and exponentials 8
4 Machines for machine learning 9
What matters 9
The reference machine 9
Other machines 10
Machine time 10
Keeping a long run alive 11
5 Setup 12
What you need 12
Python and uv 12
The self-test 13
Data and models 13
Troubleshooting 14
6 Reference library 15
Books 15
Documentation 15
Courses and talks 15
Papers by unit 15
7 Progress tracker 17

AI Foundations, First edition, October 2026. © 2026 O'Neill Group LLC. PyTorch is a trademark of the Linux Foundation. Apple, Apple Silicon and Metal are trademarks of Apple Inc. NVIDIA and CUDA are trademarks of NVIDIA Corporation. Vulkan is a registered trademark of the Khronos Group Inc. Hugging Face is a trademark of Hugging Face, Inc. All other trademarks belong to their owners. The models and datasets the course uses keep their own licences, listed in the Handbook.

PART 1

The course

AI Foundations teaches modern machine learning from first principles, to programmers who know Python well and have never trained a model. It starts with arrays of numbers and the cost of arithmetic on them, builds an automatic differentiation engine, and uses it to derive and train the networks that the rest of the field is built from. It then builds a language model from an empty file, pretrains it, studies how its loss scales and what it learns inside, turns an open-weight model into an assistant by the methods the large laboratories use, and finally makes all of it fast, down to GPU kernels and an inference engine in C++.

The course is practical, and it is honest about what it shows. Every chapter has labs with measurable done-when criteria. Every listing in these volumes is taken from companion code that runs and checks its own results, usually against PyTorch or against a calculation done another way, and every console listing and plot is real output from those programs. All of it was run on an Apple M2 Pro with 32 GB of memory, and the text says wherever other hardware behaves differently. Nothing is trained on a rented cluster; the largest run takes just under five hours on that laptop.

What you will have built

- **A deep-learning library of your own.** `tiny`, an automatic differentiation engine and tensor library of about 650 lines, checked operation by operation against central differences and against PyTorch to the last bit, with the optimisers in common use and networks trained on MNIST.
- **The classic results, reproduced.** Initialisation measured through thirty layers, overfitting cured four ways, double descent, a 30-layer network made trainable by normalisation and residual connections, a convolutional network on CIFAR-10, and an encoder trained by contrastive learning.
- **A language model from an empty file.** A byte-level tokeniser that encodes text exactly as SmolLM2's does; a transformer with rotary embeddings, grouped-query attention and SwiGLU; a 28-million-parameter model pretrained on 300 million tokens of children's stories; scaling laws that predict a run's loss before it is trained; every decoding strategy in use; and the induction heads inside your model, found and switched off.
- **An assistant.** SmolLM2's open weights loaded into your own model and matched to the reference implementation; supervised fine-tuning, LoRA, preference optimisation with DPO, and reinforcement learning with GRPO; an evaluation harness with honest confidence intervals; retrieval over a document collection with an HNSW index you wrote; and an agent that calls tools.
- **A fast stack.** Your machine's roofline, a profiled and budgeted training step, the numerics of low precision, matrix multiplication, softmax and FlashAttention written as GPU kernels in Metal or Vulkan, data, tensor and pipeline parallelism, an inference engine in C++, 8- and 4-bit quantisation, and a server with continuous batching, a paged KV cache and speculative decoding.

Four units

Unit	Chapters	Hours
1 Learning Machines	Tensors and what they cost; gradients by machine; models as probability; optimisers; neural networks; generalisation; training deep networks; representations	46
2 Language Models	Tokenisation; next-token prediction; attention; the transformer; pretraining; scaling laws; decoding; inside the model	50
3 From Base Model to Assistant	Open weights; supervised fine-tuning; LoRA; preferences; reinforcement learning with verifiable rewards; evaluation; retrieval; tools and agents	52
4 Making It Fast	The machine; profiling and memory; numerics; kernels; parallel training; an inference engine in C++; quantisation; serving	52

The hours are study time: reading, labs, problem sets and the checklist. The whole course is about 200 hours; at ten hours a week it takes about five months. Training runs add machine time on top, most of it unattended; Part 4 lists it chapter by chapter. Hours are a guide; done-when criteria are the contract.

Four threads

Four threads run through the units, and most chapters advance more than one.

- **Build it, then use the library.** Each idea is implemented from scratch before its library version is used: `tiny` before PyTorch's autograd (Chapter 1.2), optimisers before `torch.optim` (1.4), attention in NumPy before PyTorch's kernel (2.3), a tokeniser before Hugging Face's (2.1), LoRA, DPO and GRPO before any training library (Unit 3), FlashAttention before the vendor's (4.4). Each is checked against the library it replaces.

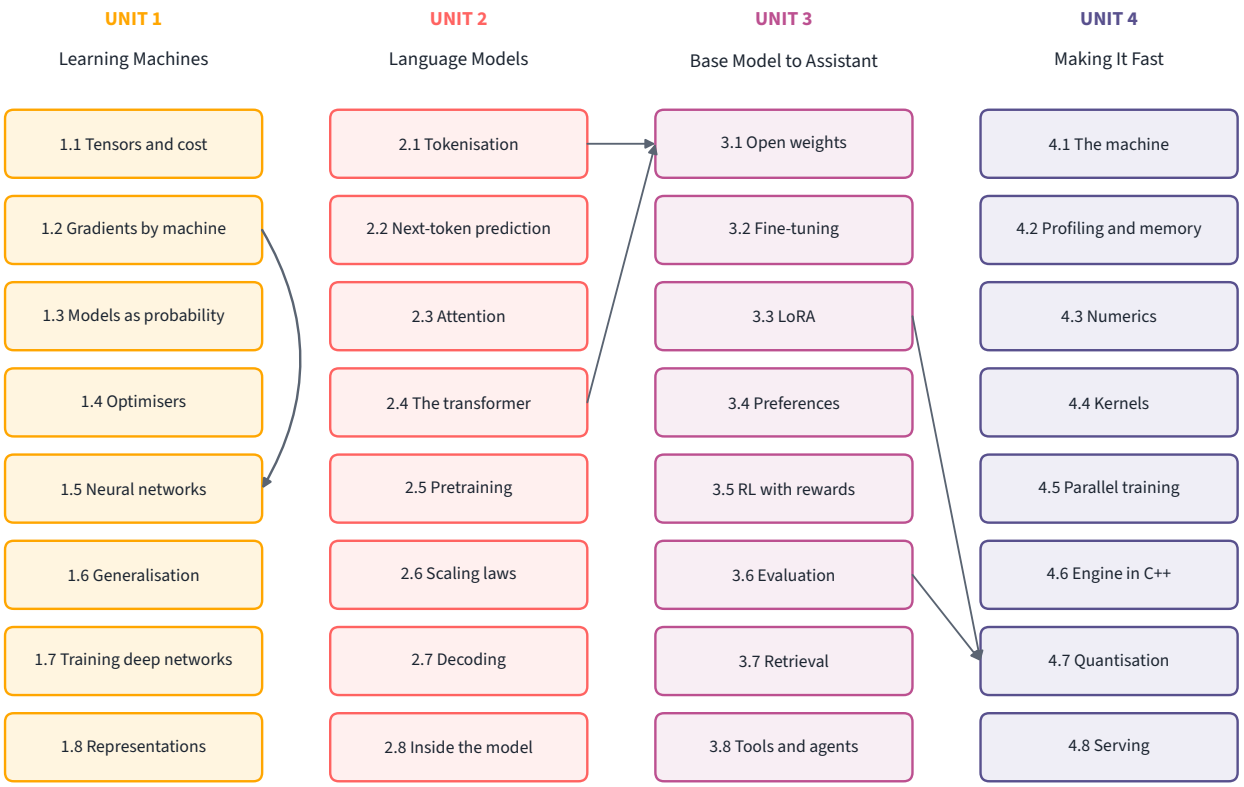


Figure 1.1. The course map. Each unit builds on the units before it, and the transformer and sampler of Unit 2 run through Units 3 and 4. Arrows mark other code that a later chapter reuses: `tiny`, the tokeniser and the transformer, and the 4-bit weights and evaluation of Unit 3.

- **Probability.** Losses as likelihoods (1.3), language modelling as compression (2.2), sampling as a design choice (2.7), preferences as a probabilistic model (3.4), and evaluation as statistics (3.6).
- **Scale.** The cost of arithmetic (1.1), what the GPU is good at (1.5), throughput in tokens per second (2.5), scaling laws (2.6), the KV cache's memory (2.7), and all of Unit 4.
- **Measurement.** Every claim has a number behind it, and every number has a run behind it that you can repeat: gradient checks, learning curves, validation losses, win rates with confidence intervals, roofline placements and latency distributions.

Who the course is for

The course assumes that you program fluently in Python: functions, classes, comprehensions, the standard library, a virtual environment and a terminal. It uses NumPy from the first chapter and introduces PyTorch as it goes. Chapters 4.6 and 4.7 and the Vulkan track of Chapter 4.4 are in C++, at the level of a programmer who has written a few thousand lines of it; the rest of the course needs none.

It assumes school mathematics: algebra, functions and graphs, logarithms and exponentials, and derivatives of simple functions. Part 3 of this Handbook summarises the linear algebra, calculus and probability the course uses, and says where each first appears. It assumes no knowledge of machine learning at all. Readers who have trained networks before will move faster through Unit 1, and should use the skip tests at the start of each chapter to decide what to skim.

The volumes and the code

The course is five volumes: this Handbook and four units. The companion code is a single Python project, managed with `uv`, with one package per unit, `u1` to `u4`, and a helper package, `aif`, that the units build up and share: the data downloads, `tiny`, the tokeniser, the transformer, the training loop and the evaluation harness. The C++ inference engine and the Vulkan kernels live beside it in `engine` and `kernels`. It arrives as an archive beside the volumes; Part 5 explains how to set it up.

Conventions

- **Versions.** The code uses Python 3.12 and PyTorch 2.14. Every model and dataset is pinned to an exact revision on the Hugging Face Hub, so that every number in these volumes can be reproduced from the same bytes.
- **Spelling.** British spelling in prose, library names verbatim: normalisation in the text, `LayerNorm` in code.
- **Numbering.** Chapters are numbered unit.chapter, so Chapter 3.2 is the second chapter of Unit 3. Labs are numbered unit.chapter.lab, and figures and tables unit.chapter.number.
- **Listings.** Each listing names the file it comes from, relative to the code folder, and shows a region marked in that file. Console listings begin with the command that produced them, run from the code folder. Long lines are wrapped to fit the page.
- **Units of measurement.** Losses are in nats unless stated otherwise; Chapter 1.3 explains nats, bits and perplexity. FLOPs count a multiplication and an addition as two. Time is in hours for study and in seconds for runs.
- **Platform.** "The M2 Pro" is the Apple M2 Pro, with 32 GB of memory, on which this course's code was run, using PyTorch's Metal backend, MPS. Commands assume a POSIX shell; on Windows, use WSL or the equivalent in PowerShell.

PART 2

How to work

The anatomy of a chapter

Every chapter has the same parts, in the same order.

- **The header** gives the study time, what you build, and what the chapter needs: earlier chapters, tools, hardware.
- **Why it matters** says what the chapter is for, in the course and in real machine-learning work.
- **The skip test** is four questions. If you can answer them all without looking anything up, skim the core ideas and do the labs the test names. If any is hard, work through the whole chapter.
- **Core ideas** teaches the material, in order, from first principles. Listings come from the companion code, and console listings from real runs. Callouts mark what to watch for: *Key idea* for the central point, *Derivation* for the mathematics worked in full, *Pitfall* for mistakes that are easy to make, and *In PyTorch* for how the library does what the chapter built.
- **Reading** lists the primary sources: the papers that introduced the ideas, book chapters and documentation. Read at least the abstract, method and main results of each paper with the chapter.
- **Labs** are the heart of the course. Each has a goal, a time, the code it starts from, numbered steps, *done-when* criteria and the evidence to keep.
- **The problem set** tests understanding without a computer.
- **The completion checklist** lists what you should now be able to do. Tick it honestly.

Each unit ends with a sign-off page: a table of its chapters and a short unit review, a set of tasks to do from memory.

Labs and done-when criteria

A lab is finished when every done-when criterion holds, not when the steps have been followed. The criteria are measurable on purpose: a check passes, a loss falls below a value, an accuracy lies within a range, an explanation fits in two sentences. If you cannot meet one, the lab has found something you do not yet understand, which is what labs are for.

The times are what the lab takes a careful engineer who has done the reading, not counting unattended training. Some will take longer on your machine, especially the first time a tool misbehaves; budget for it.

The notebook and the evidence

Keep a notebook, on paper or in plain text files in a repository: for every lab, the date, what you did, what you measured, what surprised you, and the evidence each lab asks for. Keep your code in version control with one commit per lab, so that you can return to any state.

Machine learning makes this habit more important than in most programming. A training run is an experiment with many settings, and its result depends on all of them. Without a record of the settings, the result cannot be explained or repeated, and in a month it will not be remembered.

Running experiments

The companion code follows rules that make experiments trustworthy, and your own code should follow them too.

- **Every program checks itself.** It verifies its result against an independent calculation, a reference library or an invariant, prints what it found, and exits with a non-zero status if a check fails.
- **Seeds are fixed.** Every program seeds its random number generators, so a rerun on the same machine gives the same numbers, or very nearly: some GPU operations are not deterministic, and the text notes where that matters.
- **Settings are explicit.** Programs take options such as `--lr 2e-3` or `--epochs 8`, and print or log what they used. Chapter 2.5's training program, the longest run, writes a log of every setting and every measurement to `runs/<name>/log.jsonl`; the others print theirs.
- **One change at a time.** When comparing two settings, change only one thing between runs, and keep a baseline run you trust.
- **Small first.** Before a long run, run the same program for a few steps with a tiny model, to find bugs in minutes instead of hours. `u2.pretrain --preset tiny` exists for exactly this.
- **Long runs resume.** A run that takes hours saves checkpoints and can be restarted with `--resume` after an interruption.

Measuring properly

Many labs ask for timings. Computers are hard to time well, and a careless measurement is worse than none, because it looks like knowledge.

- **Synchronise before you stop the clock.** GPU work runs asynchronously: a PyTorch call returns as soon as the work is queued. `aif.device.sync` waits for it to finish, and every timing in the course calls it.
- **Warm up, repeat, and report the median.** The first run of anything includes compilation, memory allocation and clock ramp-up. `aif.timing.median_time` runs a function several times and reports the median.
- **Measure on an idle machine.** A training run in the background, a video call or a browser full of tabs distorts every number. Laptops also change speed with their power source and temperature: measure on mains power.
- **Name the machine.** Record the processor, the GPU, the memory and the library versions with every number. `uv run -m aif.selftest` prints them all.

Reading papers

The field moves through papers, and the course asks you to read about fifty of them. A first pass takes thirty minutes: the abstract, the introduction's last paragraph, the method's main equations, and the main table of results. Ask what the paper claims, what evidence supports it, and what it compared against. A second pass, for the papers a chapter is built on, works through the method with a pen. The reading lists say which is which by placing the essential papers first.

Getting unstuck

When something goes wrong, work through these in order.

1. **Read the error, all of it.** Python's traceback names the line; PyTorch's shape errors name both shapes. Most problems end here.
2. **Print shapes and values.** Most bugs in machine-learning code are shapes that broadcast unintentionally or values that are not what you think. Check both at every step of the failing computation.
3. **Make it smaller.** Remove everything that does not affect the failure until what remains fits on a screen: a tiny model, one batch, one step. The cause is usually obvious, and if not, it is a good question to ask someone.
4. **Check against a reference.** Every component in the course has a reference: central differences, PyTorch, Hugging Face. Compare yours with it on the same inputs, one component at a time.
5. **Debug the training, not just the code.** If the code runs but the model trains badly, use the method of Chapter 1.7: check the initial loss, overfit one batch, and read the gradient and activation statistics.

PART 3

The maths you need

The course uses three areas of mathematics: linear algebra, for the shapes of data and models; calculus, for training; and probability, for losses, sampling and evaluation. This part summarises what each chapter assumes, so that you can check yourself before you start, and says where each idea is first used. It is a reminder, not a course: if a section is new to you, the reading list at the end of the part gives a gentle source for each.

Vectors and matrices

A *vector* is a list of n numbers, $x = (x_1, \ldots, x_n)$, and a *matrix* an $m \times n$ table of them. The course writes vectors as columns, so a matrix W of shape $m \times n$ maps a vector x of length n to Wx of length m, with $(Wx)_i = \sum_j W_{ij} x_j$. In code, a batch of vectors is a matrix with one vector per *row*, and a linear layer computes `x @ W`, so the shapes there read the other way round; Chapter 1.5 is careful about which convention each listing uses.

- **The dot product** $x \cdot y = \sum_i x_i y_i$ measures how far two vectors point the same way; the *norm* $\|x\| = \sqrt{x \cdot x}$ is a vector's length, and $x \cdot y = \|x\|\|y\| \cos\theta$. Attention (Chapter 2.3) and retrieval (3.7) are built on dot products.
- **Matrix multiplication** $C = AB$ needs the inner sizes to agree: an $m \times k$ matrix times a $k \times n$ matrix is $m \times n$, with $C_{ij} = \sum_p A_{ip} B_{pj}$. It is associative but not commutative, and $(AB)^T = B^T A^T$. It costs $2mkn$ floating-point operations (Chapter 1.1).
- **The transpose** A^T swaps rows and columns; a matrix with $A = A^T$ is *symmetric*.
- **Eigenvectors** of a square matrix H satisfy $Hv = \lambda v$: H only stretches them, by the *eigenvalue* λ. A symmetric matrix has a full set of perpendicular eigenvectors. Chapter 1.4 uses them to explain why some loss surfaces are hard to descend.
- **Rank and the pseudo-inverse.** The rank of a matrix is the number of independent directions it can produce. A system $Ax = b$ with more unknowns than equations has many solutions, and the *pseudo-inverse* A^+ picks the shortest one; Chapter 1.6 uses it to explain double descent, and Chapter 3.3 uses low rank to make fine-tuning cheap.

Derivatives and the chain rule

The *derivative* $f'(x)$ is the slope of f at x: for a small change δ, $f(x + \delta) \approx f(x) + f'(x)\delta$. The derivatives the course uses most are

$$(x^n)' = nx^{n-1}, \quad (e^x)' = e^x, \quad (\ln x)' = \frac{1}{x}, \quad (\tanh x)' = 1 - \tanh^2 x, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x)),$$

where $\sigma(x) = 1/(1 + e^{-x})$. A function of many variables has a *partial derivative* for each, $\partial f / \partial x_i$, the slope when only x_i changes, and the vector of them all is the *gradient* ∇f, which points in the direction of steepest increase. Training moves parameters a little against the gradient of the loss (Chapter 1.4).

The *chain rule* differentiates compositions: if $y = g(u)$ and $u = h(x)$, then $\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$. When x affects y through several paths, the contributions add. Automatic differentiation (Chapter 1.2) is nothing but the chain rule, applied systematically to a program. For functions from vectors to vectors, the matrix of all partial derivatives is the *Jacobian*, and the chain rule multiplies Jacobians; Chapter 1.7 uses this to explain why gradients vanish in deep networks.

A function's *second* derivatives describe its curvature. For a function of many variables they form the symmetric *Hessian* matrix, whose eigenvalues say how sharply the function curves in each direction (Chapter 1.4).

Probability

A *random variable* takes values according to a *distribution*. The course uses four.

- **Bernoulli**: a yes or no with probability p of yes. Binary classification (Chapter 1.3).
- **Categorical**: one of K outcomes with probabilities $p_1, \ldots, p_K$ summing to one. Classification, and the next token of a language model (Unit 2).
- **Gaussian** (normal): a continuous value with mean μ and standard deviation σ, density $1/\sqrt{2\pi\sigma^2}\,\exp\!\left(-(x-\mu)^2/(2\sigma^2)\right)$. Noise in regression (Chapter 1.3) and initial weights (1.5).
- **Uniform**: every value in a range equally likely. Random initialisation and sampling.

The *expectation* $E[X] = \sum_x xP(x)$ is the mean value, and is linear: $E[aX + bY] = aE[X] + bE[Y]$. The *variance* $\mathrm{Var}(X) = E\left[(X - E[X])^2\right]$ measures spread; its square root is the standard deviation. For *independent* variables, variances add: $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$. Chapter 1.5 derives initialisation from that rule, and Chapter 1.6 the precision of a test score.

Conditional probability $P(A \mid B) = P(A, B)/P(B)$ is the probability of A once B is known, and *Bayes' rule* turns it round: $P(A \mid B) = P(B \mid A)P(A)/P(B)$. A language model is a conditional distribution, $P(\text{next token} \mid \text{the tokens so far})$, and the probability of a whole text is the product of these conditionals, the chain rule of probability (Chapter 2.2).

The *sample mean* of n independent observations estimates the expectation, with a standard error of $\sigma/\sqrt{n}$: halving the uncertainty needs four times the data. Chapter 3.6 builds evaluation on this fact and on the bootstrap, which estimates uncertainty by resampling.

Logarithms and exponentials

The course takes logarithms constantly, because probabilities multiply and logarithms turn products into sums: $\log(ab) = \log a + \log b$. Natural logarithms, base e, are the default; a quantity in natural-log units is in *nats*, and dividing by $\ln 2$ converts it to *bits*. The exponential e^x grows fast enough to overflow single-precision floating point above $x \approx 88.7$, which is why Chapter 1.3 computes softmax and log-probabilities with care.

Idea	First used
Matrix multiplication, its cost and shapes	Chapter 1.1
Chain rule, gradients, Jacobians	Chapters 1.2 and 1.7
Maximum likelihood, Gaussian and categorical distributions	Chapter 1.3
Hessian, eigenvalues, condition number	Chapter 1.4
Variance of sums of independent variables	Chapter 1.5
Pseudo-inverse, rank	Chapters 1.6 and 3.3
Conditional probability and the chain rule of probability	Chapter 2.2
Bradley–Terry model, log-odds	Chapter 3.4
Standard error, confidence intervals, the bootstrap	Chapters 1.6 and 3.6

PART 4

Machines for machine learning

The course runs on one computer. This part says what matters about a machine for machine learning, what the reference machine is, what other machines can do, and how long each chapter's programs take, so that you can plan the long runs.

What matters

Three properties of a machine decide what it can train and how fast, and Chapter 4.1 measures them properly.

- **Memory capacity** decides what fits. Training with AdamW in float32 needs 16 bytes per parameter before activations, Chapter 4.2's memory equation: about 0.5 GB for the course's 28-million-parameter model and 5.8 GB for full fine-tuning of SmolLM2-360M, with activations on top. Apple Silicon shares one pool of *unified memory* between the CPU and the GPU, so a Mac with 32 GB can give most of it to the GPU; an NVIDIA GPU has its own memory, typically 8 to 24 GB on a desktop card.
- **Memory bandwidth** decides the speed of everything that does little arithmetic per byte: normalisations, element-wise operations and, above all, generating text one token at a time, which reads every weight once per token.
- **Arithmetic throughput** decides the speed of large matrix multiplications, and so of training. NVIDIA GPUs have *tensor cores* that run 16-bit products many times faster than float32; Apple's GPUs before the M5 generation have no such units, which is one reason the course trains in float32 on the M2 Pro.

Software matters as much as silicon. PyTorch's CUDA backend, for NVIDIA GPUs, is the most mature; its Metal backend, MPS, runs everything in this course, but some operations are slower than their CUDA equivalents, and an operation MPS lacks stops the program with an error naming it; setting `PYTORCH_ENABLE_MPS_FALLBACK=1` runs such operations on the CPU instead.

The reference machine

Every console listing and measurement in the course comes from a 16-inch MacBook Pro of 2023 with an Apple M2 Pro: 12 CPU cores, 8 for performance and 4 for efficiency, a 19-core GPU and 32 GB of unified memory. Its memory's published bandwidth is 200 GB/s, and its GPU's float32 peak about 6.8 TFLOP/s; Chapter 4.1 measures what programs can actually get. PyTorch reaches the GPU through MPS.

The M2 Pro is a capable machine for this course and a small one by the standards of the field: a data-centre GPU such as NVIDIA's H100 has about seventeen times its bandwidth and well over a hundred times its 16-bit matrix arithmetic. The course's techniques are the same at both scales, and Unit 4 says where the balance between them differs.

Other machines

Machine	What to expect
Apple Silicon Pro or Max, 32 GB or more	Everything, at about the times below; a Max chip, with twice the GPU cores and bandwidth, can take about half as long.
Apple Silicon with 16 GB	Units 1, 2 and 4 in full, with Chapter 2.5's run two or more times as long on a base chip. In Unit 3, fine-tune SmolLM2-135M where the text uses 360M, or use LoRA.
NVIDIA GPU with 16 to 24 GB	Everything, usually faster than the M2 Pro. On a GPU of the Ampere generation or later, Chapter 2.5's `--dtype bf16` trains several times faster than float32. The Metal track of Chapter 4.4 needs a Mac; follow the Vulkan track.
NVIDIA GPU with 8 to 12 GB	Units 1, 2 and 4, with smaller batches and gradient accumulation where memory runs out. In Unit 3, use LoRA and the 135M models.
CPU only	Unit 1 to Chapter 1.7, the n-gram models, and the C++ chapters run normally. Use the `small` preset and fewer tokens for Chapters 2.5 and 2.6, and expect Unit 3's runs to take a day or more.

Renting a single GPU by the hour from a cloud provider is a sensible alternative for the long runs of Units 2 and 3: the code runs unchanged on CUDA, and `uv sync` installs the same environment. Check the rate, and remember to stop the machine.

The C++ engine of Chapter 4.6 uses ARM's NEON instructions on Apple Silicon and other ARM processors; elsewhere it falls back to plain loops, correct but several times slower, and writing AVX2 versions is a worthwhile extension of Lab 4.6.3.

Machine time

The study hours in each chapter's header leave out time a program spends running unattended. This table lists every program that runs for more than a minute on the M2 Pro with 32 GB, rounded, as measured while the course was written, one program at a time.

Chapter	Command, after `uv run -m`	Time
1.7	`u1.deep`	1 min
1.8	`u1.cifar`	22 min
1.8	`u1.contrastive`	22 min
2.2	`u2.ngram`	3 min
2.2	`u2.mlp_lm`	10 min
2.5	`u2.pretrain`	4 h 45 min
2.5	`u2.throughput`	3 min
2.6	`u2.scaling`	3 h
2.8	`u2.probe`	2 min
3.2	`u3.sft`	1 h 20 min
3.3	`u3.lora`	1 h 15 min
3.3	`u3.lora --full --examples 5000`	40 min
3.3	`u3.lora --model HuggingFaceTB/SmolLM2-360M --int4 --examples 5000`	2 h 5 min
3.4	`u3.reward`	30 min
3.4	`u3.dpo`	1 h 10 min
3.5	`u3.grpo`	1 h 5 min
3.6	`u3.evaluate`	50 min
3.7	`u3.retrieval`	1 h 50 min
3.8	`u3.agent`	4 min
4.7	`u4.quantise --model HuggingFaceTB/SmolLM2-135M`	6 min
4.8	`u4.serve`	6 min

Chapter 2.5's run, the longest, can be stopped and resumed: `--resume` continues from the last checkpoint.

Keeping a long run alive

- **Stop the machine sleeping.** On macOS, prefix the command with `caffeinate -i`, which keeps the Mac awake until the program ends; keep a laptop on mains power with its lid open, where it cools best. On Linux, run long jobs inside `tmux` so that closing the terminal does not end them.
- **Run one GPU job at a time.** Two programs sharing the GPU each run at less than half speed, and every timing in the course assumes an idle machine.
- **Watch it.** Training programs print their progress and write a log; the GPU's load is visible in Activity Monitor's GPU History window on a Mac, and with `nvidia-smi` on NVIDIA machines.
- **Make it smaller first.** Before a long run, run the same command with a tenth of the tokens or steps and check that the loss falls. A mistake found in five minutes costs five minutes.

PART 5

Setup

This part sets up the companion code and checks your machine. It takes about an hour, most of it downloads.

What you need

- **A computer with 16 GB of memory or more.** 32 GB is comfortable for Unit 3, which fine-tunes models with up to 360 million parameters and runs one with 1.7 billion. Part 4 says what each kind of machine can do.
- **A GPU, ideally.** An Apple Silicon Mac uses its GPU through PyTorch's Metal backend, MPS; a Linux or Windows machine with an NVIDIA GPU uses CUDA. Without a GPU, Units 1 and the C++ chapters run normally and the rest runs slowly, with smaller settings.
- **About 15 GB of free disk space** for the datasets, the models and the checkpoints your runs write.
- **A terminal and an editor.** Any editor with Python support will do.
- **For Chapters 4.6 and 4.7**, a C++20 compiler, CMake 3.25 or later and Ninja. **For the Vulkan track of Chapter 4.4**, the Vulkan SDK as set up in Part 5 of Vulkan Foundations' Handbook; the Metal track needs only macOS and the companion code.

Python and uv

The course uses `uv`, a fast installer and environment manager for Python, to give everyone the same versions of everything. Install it from its website, or with Homebrew on macOS:

```
brew install uv
```

`uv` installs the right version of Python itself, so no separate Python installation is needed. Unpack the code archive, then from its folder:

```
cd ai-foundations/code
uv sync
```

`uv sync` reads `pyproject.toml` and the lock file `uv.lock`, creates a virtual environment in `.venv`, and installs Python 3.12, NumPy, PyTorch, the Hugging Face libraries used for downloads and for reference implementations, Matplotlib and SciPy, at exactly the versions this course was run with. Every program then runs through `uv run`, which uses that environment:

```
uv run -m u1.gradcheck
```

Programs are modules inside the packages `u1` to `u4`, so they are run with `-m` and a dotted name, from the `code` folder.

The project's dependencies `code/pyproject.toml`

```
[project]
name = "ai-foundations"
version = "1.0.0"
description = "Companion code for AI Foundations"
requires-python = "⩾3.12,<3.13"
dependencies = [
    "numpy⩾2.1",
    "torch⩾2.8",
    "safetensors⩾0.4",
    "huggingface-hub⩾0.30",
    "pyarrow⩾17",
    "matplotlib⩾3.9",
    "scipy⩾1.14",
    "transformers⩾4.50",
]

[tool.uv]
package = false
```

The self-test

`aif.selftest` checks the installation: it reports the versions, the processor, the memory and the device PyTorch will use, multiplies matrices on the CPU and the GPU and checks the results, runs a gradient, checks that the data folders are writable, and, with `--download`, fetches MNIST and CIFAR-10.

The self-test on the M2 Pro

```
$ uv run -m aif.selftest --download
Python 3.12.15, NumPy 2.5.3, PyTorch 2.14.1
machine: Apple M2 Pro, 32 GiB of memory, Darwin 25.6.0 on arm64
device: mps (the GPU of the Apple M2 Pro, through Metal)
matrix product on cpu: correct, 1.97 TFLOP/s at 2048 x 2048
matrix product on mps: correct, 5.43 TFLOP/s at 2048 x 2048
autograd: correct
data folder: data (writable)
runs folder: runs (writable)
MNIST: 60,000 training and 10,000 test images
CIFAR-10: 50,000 training and 10,000 test images
self-test passed
```

If the device line says `cpu` on a machine with a GPU, PyTorch cannot see the GPU: on a Mac, check that you are running an arm64 build of macOS 14 or later; on Linux, that the NVIDIA driver is installed and `nvidia-smi` works. To force a device, set `AIF_DEVICE`, for example `AIF_DEVICE=cpu uv run -m u1.deep`.

Data and models

Programs download what they need on first use, from the Hugging Face Hub, into `code/data`, at the exact revision recorded in `aif/data.py`. Nothing needs an account. Set `AIF_DATA` to put the data elsewhere, and `AIF_RUNS` for the folder where runs write checkpoints and logs.

Every download goes through one function, pinned to a revision `code/aif/data.py`

```
def fetch(repo: str, filename: str, kind: str = "dataset") → Path:
    target = DATA / repo.replace("/", "--")
    if (target / filename).exists():
        return target / filename
    path = hf_hub_download(
        repo, filename, repo_type=kind, revision=REVISIONS[repo], local_dir=target
    )
    return Path(path)
```

Dataset or model	Size	Licence	Used in
MNIST, handwritten digits	18 MB	MIT	1.3–1.7
CIFAR-10, small colour photographs	144 MB	none stated; distributed by its authors for research	1.8
TinyStories, synthetic children's stories, version 2	2.25 GB	CDLA-Sharing 1.0	Unit 2, 4
SmolLM2 135M, base and instruction-tuned	540 MB	Apache 2.0	Units 2–4
SmolLM2 360M, base and instruction-tuned	1.45 GB	Apache 2.0	Units 3, 4
SmolLM2 1.7B, instruction-tuned	3.4 GB	Apache 2.0	3.6, 3.8, 4.8
smol-smoltalk, conversations, one shard	280 MB	Apache 2.0	3.2, 3.3, 3.6, 4.8
UltraFeedback, binarised preference pairs	233 MB	MIT	3.4
ARC, science exam questions	1 MB	CC BY-SA 4.0	3.6
SQuAD 1.1, questions on Wikipedia paragraphs	16 MB	CC BY-SA 4.0	3.7, 3.8

The licences permit study and experiment. If you publish models you train on these datasets, check each licence's terms for redistribution; CDLA-Sharing and CC BY-SA ask that derived data be shared under the same terms.

Troubleshooting

Symptom	Cause and fix
`uv: command not found`	`uv` installed to a folder not on your `PATH`. Open a new terminal, or follow the installer's instruction to add it.
`ModuleNotFoundError: No module named 'aif'`	The program was run from the wrong folder or without `-m`. Run `uv run -m u1.gradcheck` from the `code` folder.
The device is `cpu` on a Mac	The Python is an x86 build running under Rosetta. Delete `.venv` and run `uv sync` from a native arm64 terminal.
`MPS backend out of memory`	Another program holds GPU memory, or a batch is too large. Close other applications, or lower the batch: `--batch` in Units 1 and 2, `--batch-tokens` in Unit 3, where `u3.lora` prints the memory it uses.
Downloads fail or stall	A proxy or firewall blocks huggingface.co. Set `HF_ENDPOINT` to a mirror, or download the files listed in `aif/data.py` by hand into `code/data`.
A program's check fails on your machine	Compare your numbers with the console listing in the chapter. Small differences in timings and in the last digits of losses are expected; a failed check is not, and is worth a careful look before you continue.

PART 6

Reference library

The course is self-contained, but it stands on a literature that you should know your way around. This part lists the books worth owning or bookmarking, the documentation the code relies on, and the papers each unit is built on. Every chapter's reading list draws on it.

Books

- **Simon Prince, *Understanding Deep Learning***, MIT Press, 2023. The clearest modern textbook, with excellent figures; free online. The companion text for Units 1 and 2.
- **Christopher Bishop and Hugh Bishop, *Deep Learning: Foundations and Concepts***, Springer, 2024. More mathematical, strong on probability; free to read online.
- **Kevin Murphy, *Probabilistic Machine Learning: An Introduction*** and ***Advanced Topics***, MIT Press, 2022 and 2023. The reference for the probabilistic view of everything; free drafts online.
- **Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning***, MIT Press, 2016. Older, but its chapters on optimisation and regularisation remain the standard account.
- **Daniel Jurafsky and James Martin, *Speech and Language Processing***, third edition draft. N-gram models, smoothing and tokenisation, for Chapters 2.1 and 2.2.
- **Mark Peter Deisenroth, A. Aldo Faisal and Cheng Soon Ong, *Mathematics for Machine Learning***, Cambridge University Press, 2020. The mathematics of Part 3, at a gentle pace; free online.

Documentation

- **The PyTorch documentation**, in particular "Autograd mechanics", "CUDA semantics", the `torch.nn` and `torch.optim` references, and the notes on numerical accuracy and reproducibility.
- **The Hugging Face documentation** for the `safetensors` format, the `tokenizers` library's model types, and the model cards of SmolLM2 and the datasets in Part 5.
- **Apple's Metal Shading Language Specification** and **the Vulkan specification**, for the two tracks of Chapter 4.4.

Courses and talks

- **Andrej Karpathy, "Neural Networks: Zero to Hero"**, a video series that builds a scalar autograd engine and a GPT, much as Units 1 and 2 do.
- **Stanford CS336, "Language Modeling from Scratch"**, a university course on the material of Units 2 to 4, with assignments at larger scale.

Papers by unit

Unit 1. Glorot and Bengio (2010) on initialisation; He and others (2015) on rectifiers; Kingma and Ba (2015), Adam; Loshchilov and Hutter (2019), AdamW; Srivastava and others (2014), dropout; Ioffe and Szegedy (2015), batch normalisation; Ba, Kiros and Hinton (2016), layer normalisation; He and others (2016), residual networks; Zhang and others (2017), rethinking generalisation; Belkin and others (2019), double descent; Chen and others (2020), SimCLR.

Unit 2. Sennrich, Haddow and Birch (2016), byte-pair encoding; Bengio and others (2003), a neural probabilistic language model; Vaswani and others (2017), "Attention Is All You Need"; Radford and others (2019),

GPT-2; Su and others (2021), rotary position embeddings; Shazeer (2020), GLU variants; Ainslie and others (2023), grouped-query attention; Eldan and Li (2023), TinyStories; Kaplan and others (2020) and Hoffmann and others (2022), scaling laws; Holtzman and others (2020), nucleus sampling; Olsson and others (2022), induction heads.

Unit 3. Allal and others (2025), SmolLM2; Ouyang and others (2022), InstructGPT; Hu and others (2022), LoRA; Dettmers and others (2023), QLoRA; Rafailov and others (2023), DPO; Shao and others (2024), DeepSeekMath and GRPO; Lewis and others (2020), retrieval-augmented generation; Malkov and Yashunin (2018), HNSW; Yao and others (2023), ReAct; Zheng and others (2023), LLM-as-a-judge.

Unit 4. Williams, Waterman and Patterson (2009), the roofline model; Micikevicius and others (2018), mixed-precision training; Dao and others (2022), FlashAttention; Rajbhandari and others (2020), ZeRO; Shoeybi and others (2019), Megatron-LM; Huang and others (2019), GPipe; Frantar and others (2023), GPTQ; Kwon and others (2023), PagedAttention and vLLM; Leviathan, Kalman and Matias (2023), speculative decoding; Yu and others (2022), Orca and continuous batching.

PART 7

Progress tracker

Every lab in the course, in order. Date each line when the lab's done-when criteria all hold and its evidence is filed, and the chapter line when its completion checklist is ticked. The sign-off page at the end of each unit is the formal record; this is the map of where you are.

Unit 1: Learning Machines

Lab	Title	Date done
Chapter 1.1 Tensors and what they cost		
Lab 1.1.1	Strides by hand	
Lab 1.1.2	Broadcasting drills	
Lab 1.1.3	What a temporary costs	
Lab 1.1.4	Your machine's ladder	
Checklist	*Every line of Chapter 1.1's completion checklist ticked*	
Chapter 1.2 Gradients by machine		
Lab 1.2.1	XOR with a scalar engine	
Lab 1.2.2	A new operation	
Lab 1.2.3	Break it on purpose	
Lab 1.2.4	What a gradient costs	
Checklist	*Every line of Chapter 1.2's completion checklist ticked*	
Chapter 1.3 Models as probability		
Lab 1.3.1	When the noise varies	
Lab 1.3.2	Two classes, two ways	
Lab 1.3.3	Stable everywhere	
Lab 1.3.4	Reading the weights	
Checklist	*Every line of Chapter 1.3's completion checklist ticked*	
Chapter 1.4 Optimisers		
Lab 1.4.1	The edge of stability	
Lab 1.4.2	Tuning momentum	
Lab 1.4.3	SGD against Adam on MNIST	
Lab 1.4.4	Range test and schedule	
Checklist	*Every line of Chapter 1.4's completion checklist ticked*	
Chapter 1.5 Neural networks		
Lab 1.5.1	Predicting the scale	
Lab 1.5.2	Width and depth	
Lab 1.5.3	When the GPU wins	
Lab 1.5.4	Inside the network	
Checklist	*Every line of Chapter 1.5's completion checklist ticked*	
Chapter 1.6 Generalisation		
Lab 1.6.1	Learning curves	
Lab 1.6.2	Tuning without leaking	
Lab 1.6.3	Memorising noise	
Lab 1.6.4	What moves the peak	

Lab	Title	Date done
Checklist	*Every line of Chapter 1.6's completion checklist ticked*	
Chapter 1.7 Training deep networks		
Lab 1.7.1	Depth, measured	
Lab 1.7.2	Taming post-norm	
Lab 1.7.3	Three mystery runs	
Lab 1.7.4	A training monitor	
Checklist	*Every line of Chapter 1.7's completion checklist ticked*	
Chapter 1.8 Representations		
Lab 1.8.1	What augmentation buys	
Lab 1.8.2	Fast to 93%	
Lab 1.8.3	Which views matter	
Lab 1.8.4	Few labels	
Checklist	*Every line of Chapter 1.8's completion checklist ticked*	

Unit 2: Language Models

Lab	Title	Date done
Chapter 2.1 Tokenisation		
Lab 2.1.1	BPE by hand	
Lab 2.1.2	Choosing a vocabulary size	
Lab 2.1.3	A tokeniser for code	
Lab 2.1.4	Tokens and letters	
Checklist	*Every line of Chapter 2.1's completion checklist ticked*	
Chapter 2.2 Next-token prediction		
Lab 2.2.1	Perplexity by hand	
Lab 2.2.2	Smoothing compared	
Lab 2.2.3	Data and order	
Lab 2.2.4	Context length	
Checklist	*Every line of Chapter 2.2's completion checklist ticked*	
Chapter 2.3 Attention		
Lab 2.3.1	Attention by hand	
Lab 2.3.2	One layer against two	
Lab 2.3.3	Without the scale	
Lab 2.3.4	The quadratic cost	
Checklist	*Every line of Chapter 2.3's completion checklist ticked*	
Chapter 2.4 The transformer		
Lab 2.4.1	Shapes everywhere	
Lab 2.4.2	Ablations at small scale	
Lab 2.4.3	Fewer key-value heads	
Lab 2.4.4	Count SmolLM2 by hand	
Checklist	*Every line of Chapter 2.4's completion checklist ticked*	
Chapter 2.5 Pretraining		
Lab 2.5.1	Your run	
Lab 2.5.2	A learning-rate sweep	
Lab 2.5.3	What keeps it stable	
Lab 2.5.4	Accumulation, checked	
Checklist	*Every line of Chapter 2.5's completion checklist ticked*	
Chapter 2.6 Scaling laws		

Lab	Title	Date done
Lab 2.6.1	The three-term law	
Lab 2.6.2	Counting parameters	
Lab 2.6.3	Smaller and longer	
Lab 2.6.4	One learning rate for all	
Checklist	*Every line of Chapter 2.6's completion checklist ticked*	
Chapter 2.7 Decoding		
Lab 2.7.1	A repetition penalty	
Lab 2.7.2	Temperature and truncation	
Lab 2.7.3	The width of the beam	
Lab 2.7.4	Many samples at once	
Checklist	*Every line of Chapter 2.7's completion checklist ticked*	
Chapter 2.8 Inside the model		
Lab 2.8.1	Previous-token heads	
Lab 2.8.2	When they form	
Lab 2.8.3	A tuned lens	
Lab 2.8.4	Steering by a direction	
Checklist	*Every line of Chapter 2.8's completion checklist ticked*	

Unit 3: From Base Model to Assistant

Lab	Title	Date done
Chapter 3.1 Open weights		
Lab 3.1.1	Inside a checkpoint	
Lab 3.1.2	A bigger model	
Lab 3.1.3	Templates matter	
Lab 3.1.4	Generalist against specialist	
Checklist	*Every line of Chapter 3.1's completion checklist ticked*	
Chapter 3.2 Supervised fine-tuning		
Lab 3.2.1	Seeing the mask	
Lab 3.2.2	Without the mask	
Lab 3.2.3	How much and how fast	
Lab 3.2.4	Your own style	
Checklist	*Every line of Chapter 3.2's completion checklist ticked*	
Chapter 3.3 LoRA		
Lab 3.3.1	The rank	
Lab 3.3.2	Where to adapt	
Lab 3.3.3	Memory, by hand and measured	
Lab 3.3.4	Two adapters, one model	
Checklist	*Every line of Chapter 3.3's completion checklist ticked*	
Chapter 3.4 Preferences		
Lab 3.4.1	Bradley–Terry by hand	
Lab 3.4.2	What the reward model likes	
Lab 3.4.3	The role of β	
Lab 3.4.4	DPO without fine-tuning first	
Checklist	*Every line of Chapter 3.4's completion checklist ticked*	
Chapter 3.5 Reinforcement learning with verifiable rewards		
Lab 3.5.1	Policy gradients on a bandit	
Lab 3.5.2	Group size	

Lab	Title	Date done
Lab 3.5.3	pass@k	
Lab 3.5.4	Hacking the checker	
Checklist	*Every line of Chapter 3.5's completion checklist ticked*	
Chapter 3.6 Evaluation		
Lab 3.6.1	The prompt matters	
Lab 3.6.2	How many questions?	
Lab 3.6.3	Calibrating the judge	
Lab 3.6.4	Contamination by paraphrase	
Checklist	*Every line of Chapter 3.6's completion checklist ticked*	
Chapter 3.7 Retrieval		
Lab 3.7.1	BM25's parameters	
Lab 3.7.2	Harder negatives	
Lab 3.7.3	HNSW's trade-offs	
Lab 3.7.4	Where answers are lost	
Checklist	*Every line of Chapter 3.7's completion checklist ticked*	
Chapter 3.8 Tools and agents		
Lab 3.8.1	A richer schema	
Lab 3.8.2	A new tool	
Lab 3.8.3	Reading failures	
Lab 3.8.4	Attacks and defences	
Checklist	*Every line of Chapter 3.8's completion checklist ticked*	

Unit 4: Making It Fast

Lab	Title	Date done
Chapter 4.1 The machine		
Lab 4.1.1	Bandwidth and the caches	
Lab 4.1.2	The shape of a product	
Lab 4.1.3	Fusion by hand	
Lab 4.1.4	Another machine on paper	
Checklist	*Every line of Chapter 4.1's completion checklist ticked*	
Chapter 4.2 Profiling and memory		
Lab 4.2.1	A profile of your own	
Lab 4.2.2	Predict, then measure	
Lab 4.2.3	Selective checkpointing	
Lab 4.2.4	The biggest batch	
Checklist	*Every line of Chapter 4.2's completion checklist ticked*	
Chapter 4.3 Numerics		
Lab 4.3.1	Where a sum stalls	
Lab 4.3.2	Training in float16	
Lab 4.3.3	Norms in low precision	
Lab 4.3.4	fp8 with finer scales	
Checklist	*Every line of Chapter 4.3's completion checklist ticked*	
Chapter 4.4 Kernels		
Lab 4.4.1	Tile sizes	
Lab 4.4.2	A one-pass softmax	
Lab 4.4.3	A faster flash kernel	
Lab 4.4.4	The other track	

Lab	Title	Date done
Checklist	*Every line of Chapter 4.4's completion checklist ticked*	
Chapter 4.5 Parallel training		
Lab 4.5.1	The cost of an all-reduce	
Lab 4.5.2	Overlapping communication	
Lab 4.5.3	ZeRO stage 2	
Lab 4.5.4	Tensor-parallel attention	
Checklist	*Every line of Chapter 4.5's completion checklist ticked*	
Chapter 4.6 An inference engine in C++		
Lab 4.6.1	Your model, in C++	
Lab 4.6.2	Batched prefill	
Lab 4.6.3	Against the bound	
Lab 4.6.4	Long contexts	
Checklist	*Every line of Chapter 4.6's completion checklist ticked*	
Chapter 4.7 Quantisation		
Lab 4.7.1	The bits curve	
Lab 4.7.2	Outliers	
Lab 4.7.3	GPTQ, examined	
Lab 4.7.4	Faster 4-bit kernels	
Checklist	*Every line of Chapter 4.7's completion checklist ticked*	
Chapter 4.8 Serving		
Lab 4.8.1	Load and latency	
Lab 4.8.2	Blocks on demand	
Lab 4.8.3	Prefix caching	
Lab 4.8.4	Speculative sampling	
Checklist	*Every line of Chapter 4.8's completion checklist ticked*	